

The break statement:

The break statement usually used with the switch-case statement; however, it can be used with a loop statement where it terminates the loop.

```
public class BreakExample {  
    public static void main(String[] args)  
    {  
        for (int counter = 1; counter <= 10; counter++)  
        {  
            if (counter==6)  
                break;  
            System.out.print(counter+" ");  
        }  
    }  
}
```



Problems Javadoc Declaration Console Progress
<terminated> BreakExample [Java Application] /Library/Java/JavaVirtualMachine
1 2 3 4 5

The continue statement:

The continue statement skips the current iteration of a loop and proceeds directly to the next iteration.

```
public class ContinueExample {  
    public static void main(String[] args)  
    {  
        for (int counter = 1; counter <= 10; counter++)  
        {  
            if (counter==6)  
                continue;  
            System.out.print(counter+" ");  
        }  
    }  
}
```



Problems Javadoc Declaration Console Progress
<terminated> ContinueExample [Java Application] /Library/Java/JavaVirtualMach
1 2 3 4 5 7 8 9 10

Type casting

The action of changing a data type into a different type is called type casting.

```
int a = 2; // a = 2
double a = 2; // a = 2.0 (Implicit)
int a = 18.7; // ERROR
int a = (int)18.7; // a = 18 (Explicit)
double a = 2/3; // a = 0.0
double a = (double)2/3; // a = 0.6666...
```

There are two types of conversion:

- 1. Implicit Type Conversion:** used when convert a smaller type to the larger type size. It is performed automatically.

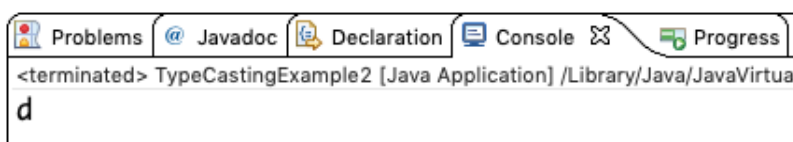
```
//Implicit type casting - automatic conversion
public class TypeCastingExample1 {
    public static void main(String[] args) {
        char ch = 'A'; // 16 bits
        // type casting
        int num = ch; // 16 bits to 32 bits
        System.out.print(num);
    }
}
```



```
<terminated> TypeCastingExample1 [Java Application] /Library/Java/JavaVirtual
65
```

- 2. Explicit Type Conversion:** used when convert a larger type to the smaller type size.

```
//Explicit type casting - no automatic conversion
public class TypeCastingExample2 {
    public static void main(String[] args) {
        int num = 100; // 32 bits
        // type casting
        char ch = (char)num; // 32 bits to 16 bits
        System.out.print(ch);
    }
}
```

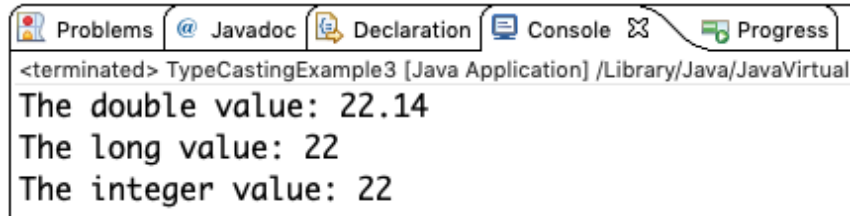


```
<terminated> TypeCastingExample2 [Java Application] /Library/Java/JavaVirtual
d
```

```

public class TypeCastingExample3 {
    public static void main(String[] args){
        double doublevalue = 22.14;
        long longvalue = (long)doublevalue;
        int intvalue = (int)longvalue;
        System.out.println("The double value: "+doublevalue);
        System.out.println("The long value: "+longvalue);
        System.out.println("The integer value: "+intvalue);
    }
}

```



```

<terminated> TypeCastingExample3 [Java Application] /Library/Java/JavaVirtual
The double value: 22.14
The long value: 22
The integer value: 22

```

Mathematical Functions in Java

In Java, the class Math provides pre-written mathematical functions to perform various math calculations. To use a mathematical function in a program:

Math.methodName(argument(s))

Class Math is implicitly imported by the compiler, so there is no need to import class Math to use its methods. The following table summarizes several Math class methods:

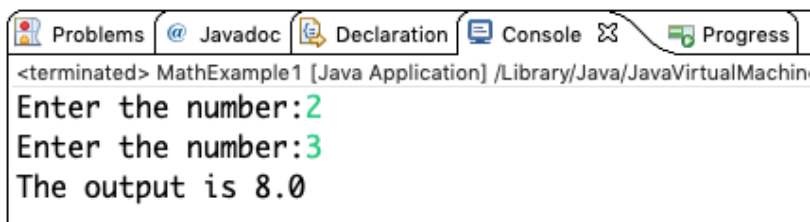
Method Name	Description	Example
abs(x)	The absolute value of x	abs(23.7) is 23.7 abs(0.0) is 0.0 abs(-23.7) is 23.7
ceil(x)	Rounds x to the smallest integer not less than x	ceil(9.2) is 10.0 ceil(-9.8) is -9.0
floor(x)	Rounds x to the largest integer not greater than x	floor(9.2) is 9.0 floor(-9.8) is -10.0
sqrt(x)	The square root of x	sqrt(900.0) is 30.0
log(x)	The natural logarithm of x (base e)	log(8) is 2.079441541
exp(x)	The exponential function base the natural logarithm	exp(1.0) is 2.71828 exp(2.0) is 7.38906
sin(x)	The sine of x (x in radians)	sin(0.0) is 0.0
cos(x)	The cosine of x (x in radians)	cos(0.0) is 1.0
tan(x)	The tangent of x (x in radians)	tan(0.0) is 0.0
max(x, y)	The maximum of two numbers	max(2.3, 12.7) is 12.7 max(-2.3, -12.7) is -2.3
min(x, y)	The minimum of two numbers	min(2.3, 12.7) is 2.3 min(-2.3, -12.7) is -12.7
pow(x, y)	The first parameter (x) raised to the power of the second parameter (y)	pow(2.0, 7.0) is 128.0 pow(9.0, 0.5) is 3.0

For examples,

```
double num1= 45.78;
System.out.println(Math.ceil(num1));           // return 46.0
System.out.println(Math.floor(num1));          // return 45.0
System.out.println(Math.min(5,6));             // return 5
```

//Java program to calculate the power.

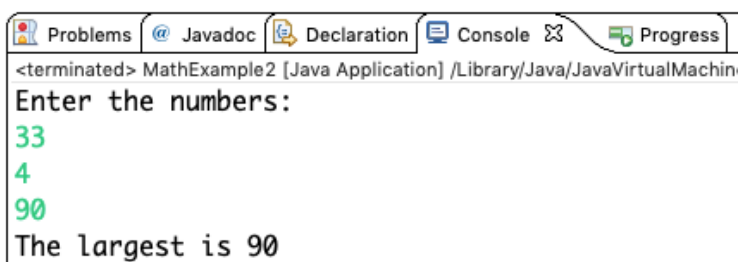
```
import java.util.Scanner;
public class MathExample1 {
    public static void main(String[] args){
        int num,power;
        Scanner input = new Scanner(System.in);
        System.out.print("Enter the number:");
        num = input.nextInt();
        System.out.print("Enter the number:");
        power = input.nextInt();
        input.close();
        System.out.println("The output is "+ Math.pow(num, power));
    }
}
```



Problems Javadoc Declaration Console Progress
<terminated> MathExample1 [Java Application] /Library/Java/JavaVirtualMachin
Enter the number:2
Enter the number:3
The output is 8.0

//Java program to find the largest of three numbers.

```
import java.util.Scanner;
public class MathExample2 {
    public static void main(String[] args){
        int num1,num2,num3,largest;
        Scanner input = new Scanner(System.in);
        System.out.println("Enter the numbers:");
        num1 = input.nextInt();
        num2 = input.nextInt();
        num3 = input.nextInt();
        input.close();
        largest=Math.max(num3, Math.max(num1, num2));
        System.out.println("The largest is "+ largest);
    }
}
```



Problems Javadoc Declaration Console Progress
<terminated> MathExample2 [Java Application] /Library/Java/JavaVirtualMachin
Enter the numbers:
33
4
90
The largest is 90